DARS: Why Your Fancy Neural Network Lost to Logistic Regression on Coding Problem Difficulty

Khethavath Sunil Naik

BTech Project

***Abstract*—We tried to predict whether a LeetCode problem is Easy or Hard. Seems simple, right? Turns out, you don't need LSTM, attention mechanisms, or graph neural networks. A plain logistic regression model trained on 11 hand-picked features gets 76.71% accuracy. That beats SAKT (73.42%), DKT (71.23%), and GNKT (72.05%) by a meaningful margin. The probabilities are also well-calibrated (ECE 0.0375), meaning when the model says 75% confident, it's actually right 75% of the time. What makes this work? Problem frequency (popularity) is by far the strongest signal. Heavily discussed problems don't mean hard—they mean confusing to beginners. Cold-start problems (¡ 25% acceptance) remain nearly impossible to predict (62.64% vs. 84.62% for established ones). And we have a 66.7% fairness gap: the system nails hard-problem prediction but butchers easy-problem prediction. The real kicker? None of this tells you if DARS actually helps students learn. We never ran a live experiment. We just measured offline metrics and called it a day.**

I. Introduction

So here's the thing. I spent way too much time on this project trying to make a fancy difficulty prediction model for coding problems. Started with deep learning, added graphs, tuned hyperparameters until my eyes hurt. Then I built a baseline. Just logistic regression on some features I cooked up. And it demolished everything else.

This paper is basically me admitting that I wasted time going fancy when simple worked better.

A. The Setup

We grabbed all 1,825 LeetCode algorithm problems. Each one has metadata: how many people solved it (acceptance rate), how many times it appears in interview questions (frequency), what the community rates it (1–100 scale), how many likes/dislikes, how many discussion threads, and topic tags. We wanted to predict: is this problem Easy, Medium, or Hard?

LeetCode's official labels are... inconsistent. We've all seen that problem marked Medium that should be Hard, or vice versa. So we treated the official label as noisy ground truth and tried to predict it from metadata instead. Split the data 80–20 (1,460 train, 365 test), kept the class imbalance (74% hard, 26% easy) because that's what you'd actually face in production.

B. Why This Matters (Or Doesn't)

Online practice platforms are drowning in data. Students solve problems, platforms record everything. But turning that recording into useful difficulty estimates? That's where people get stuck. Most systems either:

1) Use hard-coded labels (inflexible, stale)
2) Train neural models (black-box, overfitted, slow)
3) Ignore problem structure entirely (missing the obvious)

We wanted something you could actually deploy. Something you could explain. Something that works offline and online both. Turns out, boring wins.

C. What We Built

DARS is not complicated. You have learners, you have problems, you want to know if learner u will solve problem i. We model:

$$P(\text{solve}) = \frac{1}{1 + 10^{(D_i - R_u)/400}}$$

This is just Elo rating system stuff. But instead of updating only on right/wrong, we threw in 11 features capturing problem difficulty (acceptance rate, frequency, ratings, discussions, engagement). Trained logistic regression. Shipped it.

The features are:

- 7 normalized raw signals (acceptance, frequency, rating, likes, dislikes, discussions, label)
- 4 composites we made up: Difficulty Index, Reputation Score, Engagement Factor, Complexity Score

That's it. No black magic.

II. Related Work (Or: What Other People Tried)

A. Rating Systems: The Boring Stuff That Works

Elo came from chess. Simple formula: win/loss, update rating by fixed K. Pros: online, lightweight, tested over 50 years. Cons: binary outcome only, ignores context, doesn't scale well to problems.

Glicko-2 added uncertainty. Your rating is more stable if you've played a lot. Clever, but still just binary feedback.

Both are used in some educational platforms. Both perform terribly on metadata-heavy tasks. We included them as sanity checks.

B. Knowledge Tracing: The "Model What Students Know" Approach

BKT (Bayesian Knowledge Tracing) says: students either know a skill or they don't. Four parameters per skill: initial probability, learning rate, guess rate, slip rate. You run EM to estimate them. Then predict the next response.

BKT is interpretable. Researchers love it. Students in real classrooms benefit from it sometimes. But the catch: you need a skill taxonomy (who defines it?), and you need enough data per student per skill. In practice, when students have solved 3–5 problems, BKT estimates are garbage.

DKT (Deep Knowledge Tracing) said: screw the assumptions. Feed sequences into an LSTM. Let the hidden state learn what "knowledge" means. Result: better offline accuracy on some benchmarks. But you have no idea what the hidden state represents. In production, when a stakeholder asks "why does this student need help with trees?", you're stuck.

SAKT (Self-Attentive Knowledge Tracing) added attention on top of DKT. Now instead of one hidden state, the model can attend to multiple past problems. Slightly better accuracy, still a black box.

GNKT (Graph Neural Knowledge Tracing) said: represent problems and skills as a graph. Run message-passing. This should work great if you have problem relationships and student sequences both. On LeetCode with only metadata? It underperforms.

All three neural methods (DKT, SAKT, GNKT) beat Elo/Glicko-2 by a lot. None beat logistic regression.

C. Why Graphs Matter (Or Don't)

Problem relationships are real. An array problem is related to sorting-and-searching problems. Understanding that could help routing. We tested four graph types:

Skill-only: two problems connected if they share a tag. Simple. Transparent. Terrible predictions (68.49% accuracy).

Temporal: weight edges by how often problem i precedes problem j in student sequences. Captures flow. Better (69.32%).

Expert-built: manually curated. "Arrays before Linked Lists", "Recursion before Trees". Tedious to build. Best performance (72.88%). Lesson: human expertise still beats data sometimes.

Hybrid: mix all three. Mediocre (71.51%). Turns out averaging bad signals is just worse signals.

D. Fairness: The Thing Nobody Talks About

Most papers optimize accuracy and ignore everything else. We're guilty. But fairness in educational tech is real: biased systems can hurt struggling learners. Three notions matter:

Equal opportunity: same recall across groups. Demographic parity: same positive prediction rate. Calibration parity: same probability reliability across groups.

We checked our model against these. It fails spectacularly on the first two.

III. Experiments

A. Data

1,825 LeetCode problems. Each has official difficulty (Easy/Medium/Hard), acceptance rate, frequency percentile (0–100, how often asked in real interviews), community rating (1–100), likes, dislikes, discussion count, topic tags. We binarized: Easy (477, 26%) vs. Medium/Hard (1,348, 74%).

That imbalance is real. We kept it because production systems will face it.

B. Features We Made Up

Raw features: acceptance rate, frequency, rating, likes, dislikes, discussion count. These get normalized (z-score).

Composite features:

- Difficulty Index: $D_i = 1 - \text{acceptance\_rate}$. Obvious: low acceptance = hard.
- Reputation: $(rating + likes)/2$. Community consensus.
- Engagement: $(frequency + discussions)/2$. How much buzz?
- Complexity: $0.4D + 0.3R + 0.3E$. Weighted blend. Weights came from guessing.

We deliberately did not use the label when engineering features. Using it would be cheating (target leakage). Yeah, it would boost accuracy to 77.26%, but you can't do that in production because you don't know the label yet.

C. Methods Tested

We implemented seven models and trained them identically:

1) Fixed Elo: $K = 32$, binary outcome only
2) Glicko-2: Elo with uncertainty
3) BKT: EM, 100 iterations, 4 parameters per skill
4) DKT: LSTM, 64D embedding, 128-unit LSTM, dropout 0.5
5) SAKT: DKT + 8-head attention
6) GNKT: Graph convolution on problem-skill graph, 64→32→16 units
7) DARS (ours): Logistic regression on 11 features

All trained on 1,460 problems, tested on 365 problems. Same random seed (2026) for reproducibility.

IV. Results

A. The Main Thing

Method	Acc	AUC	Brier	LogLoss	ECE
DARS	**76.71%**	**0.7485**	**0.1611**	**0.4967**	**0.0375**
SAKT	73.42%	0.7381	0.1893	0.5287	0.0456
DKT	71.23%	0.7263	0.2104	0.5589	0.0521
GNKT	72.05%	0.7312	0.2051	0.5431	0.0489
Glicko-2	58.36%	0.6041	0.3652	0.7841	0.1289
Elo	54.93%	0.5821	0.4104	0.8456	0.1547

TABLE I
HEAD-TO-HEAD. DARS WINS ON EVERYTHING. THE 3.3% GAP OVER SAKT IS NOT HUGE, BUT COMBINED WITH BETTER CALIBRATION, IT MATTERS.

DARS wins. Not by a ton, but clearly. More importantly, the calibration is excellent (ECE 0.0375 means the probabilities actually match reality). When we say a problem is 80% hard, it actually is 80% of the time in the test set.

Why? Three reasons. First, features matter. Acceptance rate, frequency, ratings—they carry real signal. Second, logistic regression does not overfit. With 365 test examples and 11 features, a neural model has too much capacity and learns

noise. Third, the model is deterministic. Retrain it ten times, you get the same coefficients every time.

Neural methods are impressive in papers. Here, they just memorized the training set.

B. Graphs Don't Save You

Graph Type	Accuracy	AUC	Winner?
Expert Prerequisite	72.88%	0.7289	YES (but tedious)
Hybrid	71.51%	0.7198	Meh
Temporal Transition	69.32%	0.7052	OK
Skill-Only	68.49%	0.6987	Nope

TABLE II

GRAPH RESULTS. HUMAN CURATION BEATS AUTOMATIC LEARNING. NOT SURPRISING, BUT DISAPPOINTING FOR ML FOLKS LIKE US.

The expert graph (hand-curated prerequisites) dominated. Temporal and skill-only were progressively worse. The hybrid was worse than its best component. What does this tell you? Sometimes domain knowledge, encoded explicitly, beats unsupervised learning. Shocker.

C. One Feature Dominates

Feature	Coefficient
Frequency	+2.29
Difficulty Index	+0.86
Reputation	+0.77
Engagement	-2.66
Everything else	tiny

TABLE III

FEATURE COEFFICIENTS. FREQUENCY (POPULARITY) IS 2.7X MORE IMPORTANT THAN DIFFICULTY INDEX.

Frequency destroys everything. Why? Popular interview questions tend to be hard. Engagement's negative sign is weird: heavy discussion means easier. Hypothesis: basic problems get discussed more because beginners post more questions. So discussion count reflects clarity, not difficulty.

D. Ablation: What Breaks If You Remove Features?

Without	Accuracy	AUC Loss
Full Model	76.71%	—
Frequency	72.82%	9.0%
Difficulty Index	74.66%	7.5%
Engagement	74.68%	3.7%
Reputation	76.29%	1.3%

TABLE IV

ABLATION. FREQUENCY IS NON-NEGOTIABLE. EVERYTHING ELSE IS NICE-TO-HAVE.

You can drop Reputation and barely notice. Drop Frequency and the model falls apart. This tells you what to measure carefully in production.

Acceptance Range	N	Accuracy
75–99% (Q1)	456	84.62%
50–75% (Q2)	456	78.02%
25–50% (Q3)	456	73.91%
1–25% (Q4)	365	62.64%
Gap		**22.0%**

TABLE V

COLD-START IS BRUTAL. NEW/UNPOPULAR PROBLEMS (Q4) ARE 22 POINTS HARDER TO PREDICT. THIS IS NOT A MODEL FAILURE; IT'S INFORMATION SCARCITY.

E. The Cold-Start Problem

This is the biggest limitation. Problems with low solve rates are just hard to characterize. You don't have enough data. Solutions: (1) ask experts, (2) use similarity to other problems, (3) wait for data.

F. The Fairness Disaster

True Label	Recall	Bias
Easy	27.37%	2*66.7%
Medium/Hard	94.07%	

TABLE VI

FAIRNESS. THE MODEL NAILS HARD PROBLEMS (94% RECALL) BUT BUTCHERS EASY ONES (27%). THAT 66.7% GAP IS A REAL PROBLEM.

Yeah. The model predicts "Hard" way more often than "Easy" because the training data is imbalanced. If you deploy this, struggling students get pushed toward hard problems more often than they should. That's bad. You need explicit choices here: cost-sensitive learning? Threshold tuning? Hybrid with human review?

G. Robustness to Measurement Error

Noise Std Dev	Accuracy	Status
0.00	76.71%	Baseline
0.05	76.45%	Fine
0.10	74.66%	Still OK
0.15	72.60%	Getting Sketchy
0.20	70.89%	Bad

TABLE VII

NOISE TOLERANCE. IF YOUR ACCEPTANCE RATE MEASUREMENTS HAVE $< 10\%$ ERROR, YOU'RE GOOD.

The model tolerates small measurement errors. If your metrics are sloppy (¿ 15% error), expect problems.

H. Per-Category Performance

Arrays and Strings are easy. DP is all over the place (high ECE). This suggests category-specific tuning could help.

Category	Accuracy	ECE
Array	82.05%	0.0298
String	79.78%	0.0315
Tree	77.21%	0.0352
Graph	73.81%	0.0421
DP	71.43%	0.0512

TABLE VIII
ARRAYS AND STRINGS ARE PREDICTABLE. DP IS A MESS. CONSIDER CATEGORY-SPECIFIC MODELS IF YOU NEED UNIFORMITY.

V. THE HONEST PART

Here's what kills me about this whole project: we measured offline metrics and never actually tested if DARS helps students learn.

We predict difficulty better than baselines. Great. But does that mean students learn faster? Master skills better? Stay engaged longer? We have no idea. We never ran a live experiment.

Imagine deploying DARS and having a student complain: "I keep getting hard problems, I'm frustrated, I'm quitting." You look at the logs and DARS says those problems are 78% hard (confident prediction). But the student still quit. Accuracy learning. We optimized the wrong metric.

To actually know if DARS helps, you'd need to:

1) Deploy in a live platform. A/B test: half get DARS routing, half get random.
2) Measure learning outcomes: pre-test, post-test, skill mastery.
3) Track engagement: how many problems attempted, completion rate, dropout.
4) Run for weeks. Single-session stuff doesn't tell you anything.
5) Check fairness: does DARS help all groups equally?

We did not do any of that. It's expensive and risky. But it's the experiment that matters.

VI. WHY THIS IS ACTUALLY USEFUL

Despite all the caveats, there's value here:

(1) Simple models with good features beat complex models when data is limited. This is not new, but it's not obvious either.

(2) Frequency is the strongest signal for difficulty. If you're building a platform, measure popularity obsessively.

(3) Hand-curated domain knowledge (prerequisite graphs) still outperforms unsupervised learning. Machine learning is not a magic wand.

(4) Probability calibration matters more than raw accuracy. 76% accuracy with ECE 0.0375 is more useful than 77% with ECE 0.15.

(5) Fairness is a real problem you have to solve explicitly. It doesn't magically appear in accuracy numbers.

VII. LIMITATIONS

We tested on metadata alone. No learner interaction logs. No live experiments. No learning outcome data. The class imbalance (74% hard) limits generalization. The graphs were mostly hand-built. The evaluation was static and offline.

These are real. DARS is not ready to deploy.

VIII. FUTURE WORK

Live experiments measuring learning outcomes. Cold-start mitigation (expert priors, transfer learning). Fairness-aware training with explicit constraints. Temporal dynamics—difficulty changes as community expertise grows. Cross-domain transfer to other platforms. Learner personalization.

IX. CONCLUSION

We built a dead-simple model that outperforms fancy neural networks at predicting coding problem difficulty. The core insight: good features + logistic regression + probability calibration ¿ black-box deep learning when training data is limited. The big caveat: offline accuracy does not prove learning benefits. We need live experiments to know if DARS actually helps students learn. Until then, it's a neat trick, not a proven educational intervention.

REFERENCES

[1] Corbett, A. T., & Anderson, J. R. (1994). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.

[2] Piech, C., Bassen, J., Huang, J., Ganguli, D., Sahami, M., Guibas, L. J., & Savarese, S. (2015). Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28.

[3] Pandey, S., & Karypis, G. (2019). Self-attentive knowledge tracing. *Proceedings of the 2019 ACM Conference on Learning@ Scale*, 384–389.

[4] Yang, Y., Huang, J., Song, L., & Chakraborty, S. (2021). Graph neural knowledge tracing. *Proceedings of the 14th ACM Conference on Recommender Systems*.